

# Evaluate RAG as Separate Components

Evaluate at four boundaries:



## CORE CONCEPT

# An Evaluation Dataset Represents Real Questions

---

A RAG evaluation example should record: Include normal, rare, multi-chunk, exact-term, paraphrase, unsupported, and access-control cases.

question;

user or tenant context;

expected relevant chunk IDs;

reference answer or required facts  
when available;

unsupported or denied expectation;

importance and category;

RUN IT AND INSPECT THE RESULT

## Retrieval Precision and Recall Measure Different Goals

```
expected = {"A", "B", "C"}  
retrieved = ["A", "B", "X", "Y"]  
hits = expected.intersection(retrieved)  
print(len(hits)/len(retrieved), len(hits)/len(expected))
```

OUTPUT

0.5 0.6666666666666666

**Precision =  $2/4 = 0.50$  Recall =  $2/3 = 0.667$**

## REASONING SEQUENCE

# Top-k and Filters Change the Candidate Set

**Frame**

tenant or user  
access;

---

1

**Transform**

document type;

---

2

**Validate**

publication status;

---

3

**Explain**

date;

---

4

At k=3: precision 0.67, recall 0.50 At k=8: precision 0.38, recall 1.00 Higher k recovers all evidence but adds more irrelevant chunks. Reranking or better queries may improve the tradeoff.

## CORE CONCEPT

# Query Rewriting Can Improve Retrieval

---

A query rewrite transforms a user's question into a search query. Possible methods: Rewriting can improve recall but may change intent. Preserve the original question and record every rewritten query.

**resolve conversation references;**

**expand abbreviations;**

**generate alternate wording;**

**split a multi-part question;**

**add domain identifiers.**

RUN IT AND INSPECT THE RESULT

## Hybrid Search Combines Exact and Semantic Retrieval

```
def rrf_score(ranks, c=60):  
    return sum(1 / (c + rank) for rank in ranks)
```

### OUTPUT

A fusion score based on rank positions rather than incompatible raw scores.

Documents found by both methods gain combined evidence.

## CORE CONCEPT

# Reranking Applies a Stronger Model to Fewer Candidates

A reranker scores query-document pairs using a model more expensive than first-stage retrieval. Pipeline: Reranking can improve ordering but adds latency and cost. It cannot recover a relevant chunk missing from the candidate set.

retrieve a broader candidate set quickly;

rerank those candidates;

keep the best few for context.

## REASONING SEQUENCE

# Answer Quality Has Several Dimensions

## Frame

**Correctness:** agreement with a reference answer or verified facts

---

1

## Transform

**Relevance:** whether the response addresses the question

---

2

## Validate

**Groundedness:** whether claims are supported by supplied context

---

3

## Explain

**Citation correctness:** whether cited passages support nearby claims

---

4

Question asks refund period and exclusions. Answer gives 14 days with support but omits annual-plan exclusions. It may be grounded and partly correct, but incomplete.



RUN IT AND INSPECT THE RESULT

# Model Judges Need Human Calibration

```
agreement = sum(j == h for j, h in zip(judge_labels, human_labels))  
print(agreement / len(human_labels))
```

OUTPUT

0.84

The 8 disagreements should be grouped by cause before trusting the judge at scale.

## QUALITY GATE

# Failure Analysis Produces the Next Experiment



source and parsing status;

---



expected and retrieved chunks;

---



ranks and filters;

---



rewrite;

---

## QUALITY GATE

# An Experiment Report Includes Quality, Latency, and Cost



corpus and ingestion version;

---



embedding model and index settings;

---



retriever, filters, k, rewrite, and reranker;

---



prompt and generation model;

---

## GUIDED LAB

# Guided Lab: Improve RAG with Evidence

01

---

**label expected chunk  
IDs and required facts**

02

---

**calculate retrieval  
precision and recall**

03

---

**compare at least  
three k values**

04

---

**test metadata  
filters**